Literary Review

Chace Nuttall & Tia Cyriac

University of Minnesota Crookston

SE 3994 Undergraduate Research

1. Ni, A., Yin, P., Zhao, Y., Riddell, M., Feng, T., Shen, R., ... & Cohan, A. (2024). L2ceval: Evaluating language-to-code generation capabilities of large language models. *Transactions of the Association for Computational Linguistics*, *12*, 1311-1329.
    a. This research evaluated the language to code generation capabilities of LLMs in order to provide a comprehensive analysis of its abilities. To conduct this research, an evaluation tool known as L2cEval was used. This was done using an evaluation framework consisting of seven tasks. Given these tasks, each LLM was evaluated based on performance and accuracy. The results of the research found that each LLM struggled with minute errors and intermediate steps. Many times the details of the final output were missing and more generation was needed. However, it was still agreed upon that most LLMs are still overall highly accurate given its broad language to code generation analysis.
2. Tian, R., Ye, Y., Qin, Y., Cong, X., Lin, Y., Pan, Y., ... & Sun, M. (2024, August). Debugbench: Evaluating debugging capability of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024* (pp. 4173-4198).
    a. In this study, the focus was on evaluating the debugging capabilities of LLMs. In order to conduct this study, an LLM debugging benchmark known as DebugBench. Given these tasks, each LLM was provided with a few major and minor bug categories in Java, C++, or Python. Evaluation of this theory was based on the number of benchmarks passed. The results of the research came to many conclusions. Some of these findings include that closed source LLMs performed lower than humans and that multiple errors proved to be a great challenge. Those that performed this study hope that the created benchmarks and the total findings will contribute to the advancement of LLMs and their debugging capabilities.
    b. **Additional Links**
        i. https://github.com/thunlp/DebugBench
        ii. https://huggingface.co/datasets/Rtian/DebugBench
3. Yeo, S., Ma, Y. S., Kim, S. C., Jun, H., & Kim, T. (2024). Framework for evaluating code generation ability of large language models. *Etri Journal*, *46*(1), 106-117.
    a. This research focuses on evaluating code generation capabilities for LLMs. In order to conduct this research they propose a framework for effectively doing this. A key part of this framework is the introduction of a new metric called *pass-ratio@n*. This metric grades the LLMs based on the granularity of accuracy on the given test cases. The results of the study found that with the proposed framework, it can be fully automated to test each capability.
4. Shehab, M. A., Wardat, M., Omari, S., & Jararweh, Y. (2024, November). Evaluating large language models for code generation: Assessing accuracy, quality, and performance. In *2024 2nd International Conference on Foundation and Large Language Models (FLLM)* (pp. 407-416). IEEE.

    a. In this study, the focus is on evaluating the accuracy, quality, and performance of LLM code generation capabilities from NLP descriptions. In order to conduct this study, various datasets were used. Each LLM was graded based on their response based on the code accuracy, quality, and the overall performance. The results of this research found the error rate between three LLMs with PaLM 2 having the highest error rate of the models evaluated.
5. Wang, J., & Chen, Y. (2023, November). A review on code generation with LLMs: Application and evaluation. In *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)* (pp. 284-289). IEEE.
    a. For this research, the focus was on reviewing twenty other existing studies under two types of analysis. The first is through application and the other is with the evaluation of code generation for each LLM. For application purposes the three grading aspects included description to code, code completion, and automatic code repair. To analyze the evaluation aspect, grading factors included functional correctness and security. The results of the research found that given the progress of LLM code generation can successfully handle complex coding problems.
6. Jin, K., Wang, C. Y., Pham, H. V., & Hemmati, H. (2024, April). Can chatgpt support developers? an empirical evaluation of large language models for code generation. In *Proceedings of the 21st international conference on mining software repositories* (pp. 167-171).
    a. In this study the focus is on evaluating the code generation capabilities of LLMs to better understand if they can effectively support software developers. To conduct this research, the researchers used a dataset from various messaging histories between developers and ChatGPT. The results of this study found that ChatGPT models have been seen as successful and shown promises in effective code generation.
7. Yadav, A., Beniwal, H., & Singh, M. (2024, November). Pythonsaga: Redefining the benchmark to evaluate code generating llms. In *Findings of the Association for Computational Linguistics: EMNLP 2024* (pp. 17113-17126).
    a. This research's focus was to evaluate the common benchmarks known as HumanEval and MBPP for their python code generation. The goal was to also analyze each benchmark's diversity and difficulty. In order to do this, a proposed new benchmark called Pythonsaga was crafted with diverse and various difficulties of programming concepts. The results of this research found that a majority of the benchmarks in HumanEval and MBPP favored simple and basic programming problems. They propose that with their own crafted benchmark, Pythonsaga, this problem can be solved.
    b. Additional Links
        i. https://github.com/PythonSaga/PythonSaga

8. Wang, Z., Zhou, Z., Song, D., Huang, Y., Chen, S., Ma, L., & Zhang, T. (2025, April). Towards understanding the characteristics of code generation errors made by large language models. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)* (pp. 2587-2599). IEEE.
    a. For this research, the primary focus was to find and analyze the code generation errors made by LLMs. The goal is to assess these errors so that researchers can understand limitations and make room for opportunities for improvement. They chose to focus on six popular LLMs (CodeGen-16B, InCoder-1.3B, GPT-3.5, GPT-4, SantaCoder, and StarCoder). In order to conduct this, the evaluation metrics that were used were from the HumanEval dataset. The results of the research found that the LLMs produced 557 incorrect code solutions given this dataset. The team uses this information to propose the use of program repair agents to combat this error in code generation.
    b. **Additional Links**
        i. https://github.com/ma-labo/defects4codellm
9. Liang, S., Jiang, N., & Tan, L. (2025, July). Can language models replace programmers for coding? REPOCOD says 'not yet'. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 24698-24717).
    a. In this study, the goal was to create a unique coding benchmark that resembles real-life coding situations better than the common datasets. The goal was to create a new benchmark with these qualities called REPOCOD for python code. Each benchmark a part of REPOCOD includes more complex scenarios to understand if LLMs can perform at a professional programmer level. The results of this study found that many of the LLMs were unable to produce repository level code during generation. However, the REPOCOD dataset provided much insight into how LLMs can improve to create more repository level code.
10. Shao, M., Basit, A., Karri, R., & Shafique, M. (2024). Survey of different large language model architectures: Trends, benchmarks, and challenges. *IEEE access*, *12*, 188664-188706.
    a. This research's focus is on conducting an in-depth analysis of various LLMs to survey and find each of their strengths and limitations. The objective was to use this information to map out the area of improvement for the LLMs. To do this, each LLM was provided various tasks for each of the five distinct categories. The results of this research found that each LLM and its associated architecture provides varying strengths and limitations. The key factors included differences in performance for the tasks and cost. Using this information, the researchers proposed that others use these evaluations as a reference in their advancements of LLMs.
11. Sharifloo, A. M., Heydari, M., Kazerooni, P., Maninger, D., & Mezini, M. (2025, November). Where do LLMs still struggle? an in-depth analysis of code generation

benchmarks. In *2025 2nd IEEE/ACM International Conference on AI-powered Software (AIware)* (pp. 249-253). IEEE.

a. In this study, the concentration was on the benchmarks used to evaluate LLMs code generation and performance. The goal was to address the gaps within popular benchmarks to understand limitations that may be overlooked because of this. To conduct this study, four popular code generation benchmarks were used to identify LLM failures. The results of this study found that there were 114 tasks LLMs struggled with and the weaknesses that are associated with it. Those parts of the research provided improvements that can be made to fix this issue.
b. **Additional Links**
    i. https://github.com/breath24/FailureBench

12. Du, M., Tuan, L. A., Ji, B., Wu, X., Qing, Y., Huang, D., ... & Ng, S. K. (2025, July). Codearena: A collective evaluation platform for llm code generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)* (pp. 502-512).
    a. Many evaluations of the code generation capabilities of LLMs rely on benchmarks for information. However, issues such as data leakage and coding limitations still occur. In order to carefully evaluate LLMs with this in mind, the researchers proposed a new type of benchmark called CodeArena. To conduct this research, CodeArena includes an unbiased assessment, a public repository of test cases and their associated answers, and automated APIs for integration. The results of this research found that CodeArena allowed for more concise solutions and avoided issues such as data leakage throughout the process.
    b. **Additional Links**
        i. https://github.com/Elfsong/Monolith
13. Fakhoury, S., Naik, A., Sakkas, G., Chakraborty, S., & Lahiri, S. K. (2024). Llm-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Transactions on Software Engineering*, *50*(9), 2254-2268.
    a. This research explores the code generation of LLMs when prompted with natural language. The goal of this study is to propose an interactive workflow called. TICODER to test and support code generation accuracy given natural language prompts. In order to do this, a user study with 15 programmers was used to implement the workflow. The results of this research found that by using this workflow, there was an improvement of 45.97% in code generation accuracy.
14. Shaikhelislamov, D., Drobyshevskiy, M., & Belevantsev, A. (2024, December). LLM-based interactive code generation: Empirical evaluation. In *2024 Ivannikov Ispras Open Conference (ISPRAS)* (pp. 1-5). IEEE.
    a. In this study, the goal was to perform an empirical evaluation of the interactive code generation capabilities of LLMs. This is done to get more of an understanding of the bugs that are presented within the LLM generated code. In

order to do this, a framework known as CodePatchLLM was used. By using CodePatchLLM and three datasets on four different LLMs, the results found that there was a reduction by 19.1% in static analyzer warnings for Java.

15. Zhang, Y., Xie, Y., Lit, S., Liu, K., Wang, C., Jia, Z., ... & Liao, X. (2025, April). Unseen horizons: Unveiling the real capability of llm code generation beyond the familiar. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)* (pp. 604-615). IEEE.
    a. In this study, the researchers worked to understand the gaps in code evaluation benchmarks. With this information, the goal was to then create a new benchmark that is able to fill this gap and provide more accurate information about the LLMs. In order to do this, a new benchmark called OBFUSEVAL was developed. The results of this research found that after obfuscation, the average passing rate from the benchmark decreased 15.3%-62.5%.
    b. **Additional Links**
        i. https://github.com/zhangbuzhang/ObfusEval
16. Lipp, S., Banescu, S., & Pretschner, A. (2022, July). An empirical study on the effectiveness of static C code analyzers for vulnerability detection. In *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis* (pp. 544-555).
    a. Lipp, Banescu, and Pretschner evaluate six static C code analyzers: Five free and open source (Flawfinder, Cppcheck, Infer, CodeChecker, CodeQL) and one anonymized commercial tool. This was tested against a benchmark of 27 free and open-source programs comprising 1.15 millions codes and 192 validated CVEs. The main motivation for the study was observing that prior evaluations rely largely on synthetic benchmark datasets whose injected bugs are typically single-instruction synthetic changes and are easier to detect than vulnerabilities occurring in practicum with such benchmarks reporting detection rates around 80% and occasionally 100% for a particular vulnerability type. A methodological contribution precedes the evaluation itself: The authors determined the appropriate code granularity for automated assessment, show that fault and manifestation locations coincide within the same function in 92% of cases, and adopt function level accordingly. Analyzer-specific vulnerability identifiers are mapped to CWE identifiers and grouped into five high-level vulnerability classes using the CWE parent-child hierarchy, so that the tools reporting related rather than identical CWEs are not unfairly penalized. Detection is assessed in four scenarios varying whether all or merely one affected function must be flagged, and whether the correct vulnerability class must also be reported. Three findings from this study were crucial. First, effectiveness on real-world code is substantially lower than synthetic benchmarks suggest: even the best-performing analyzer missed between 47% and 80% of the 192 vulnerabilities depending on

the evaluation scenario. Second, analyzer combinations materially improve coverage. The best combination detects 21% to 34% more vulnerabilities than the best single analyzer while flagging roughly 15% more functions, and tools performing poorly in isolation (Infer, Cppcheck, and CodeChecker) appeared in most of the best performing combinations. Notably, the best combination of free and open-source analyzers matched or exceeded the commercial tool in all four scenarios. That being stated, the best combinations still miss between 30% and 69% of the benchmark's vulnerabilities. Finally, detection varied sharply by weakness class: memory related classes CWE-664 and CWE-703 are found most reliably, while CWE-682,CWE-691, and CWE-707 are found least, with only 8% of CWE-691 and 14% of CWE-707 vulnerabilities detected under the strictest scenarios.

   b. **Datasets: Magma v1.1** CVE based fuzzing benchmark
   c. **Binutils 2.29 and ffmpeg n3.32 81** Non-front- ported vulnerabilities, added to reduce front-porting bias
   d. **Combined Benchmarks** 27 programs, 1,154, LoC, 55,203 functions, 192 vulnerabilities across 21 CWE types grouped into five classes (CWE-664: 117, CWE-682: 27, CWE-703: 29, CWE-707:12, CWE-691:7)
   e. **Ground truth:** Validated CVE reports, not manual labeling by authors. Only detection ratio and marked-function ratio are reported; precision is not computed, because no ground truth exist for the full set of true negatives

17. Charoenwet, W., Thongtanunam, P., Pham, V. T., & Treude, C. (2024, September). An empirical study of static analysis tools for secure code review. In *Proceedings of the 33rd ACM SIGSOFT international symposium on software testing and analysis* (pp. 691-703).
    a. Charoenwt, Thontanunam, Pham, and Treude evaluate five freely available C and C++ static application security testing tools in the context of secure code review, using a dataset of 815 vulnerability-contributing commits drawn from 92 projects and contributing to 319 exploitability vulnerabilities. The study is motivated by a limitation the authors identify in previous works: existing SAST evaluations use synthetic benchmarks or released program versions containing complete vulnerabilities, which may not represent the small incremental code changes that reviewers actually examine during development. Tool selection follows fixed criteria, free of charge, operable from the command line, and actively maintained with the preceding twelve months. This reduced the initial candidate pool of 53 tools to Cppcheck, CodeChecker, CodeQl, Flawfinder and Infer, giving a wide variety of syntactic, semantic and hybrid analysis techniques. All tools are run at developer-recommended default settings on the grounds that tool authors discourage enabling all rules, and execution was automated within isolated Docker containers. The design in this study is notable for its granularity and scenario structure. Vulnerable code is located at both file and function level rather

than line level since modified lines in fixing commits do not reliably correspond to vulnerable lines in the contributing commits. Detection is assessed under eight scenarios that vary whether at one or all vulnerable changes must be flagged and whether the warning must carry the matching vulnerability type. Both the vulnerabilities and the tool warnings are mapped to CWE pillars, the top level of the CWE hierarchy, so that tools using different identifiers can be compared. For Codechecker and Infer, which do not emit CWE identifiers, the authors extended an existing mapping and consulted official documentation, with a second author experience in software security testing validating ambiguous cases. Three results support tool selection. First, single-tool effectiveness is moderate and granularity-dependent: Flawfinder, the best performer, produces warnings in at least one vulnerable function for 89% of VCCs but in at least one vulnerable function for only 52%. Requiring the correct vulnerability type reduces detection substantially, with only 1% to 46% of VCCs receiving a type-matched warning. Second, the tools are strongly complementary. Only 52 VCCs or 6%, are detected in common by CodeChecker, CodeQL, FlawFinder, and Infer, and combining the warnings of all five tools raise function level detection to 78% an increase of 26% over the best single tool. The complementarity is categorical rather than merely incremental: Only CodeWl detects improper access control (CWE-284) and insufficient control flow management (CWE-691), while only CodeChecker detects improper check or handling of exception conditions (CWE-703). Third, substantial blind spots persist even in combination with 22% of Vccs receiving no warning in a vulnerable function from any tool, and manual inspection of 400 sampled warnings finding 76% irrelevant to the vulnerability. The study also reports computation time which ranges from an average of 20 seconds for Flawfinder to 2,702 seconds for Cppcheck, with marked increases for systems that exceed 50,000 to 100,000 lines of code, and evaluates whether ranking changed functions by warning information improves review efficiency, finding that prioritization by warning density yields the largest gains.

   b. **Datasets:** The Vulnerability Contributing Commits dataset of Iannone etc al.: VCCs identified by the SZZ algorithm from CVE fixing commits, containing repository URL, commit SHA, CVE number, and CWE item but not the actual vulnerable files or functions.

18. Liu, Z., Wang, Z., Liu, H., & Li, W. (2026, April). An Empirical Study on Static Application Security Testing (SAST) Tools for Python. In *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering* (pp. 3673-3685).
    a. Liu, Wang, Liu, and Li conducted a comprehensive evaluation of Python static application security testing tools against real-world vulnerabilities. Eight tools, DevSkim, Dlint, Bandit, Bearer, CodeQl, Pysa, Semgrep, and Snyk are selected from an initial pool of 117 candidates using five criteria: the tool must be

security-oriented rather than quality-oriented, actively maintained within the preceding two years, freely available, operable from a command-line interface, and independent rather than an aggregator of other tools' results. The selected set covers the principal detection framework, with DevSkim and Dlint relying on code pattern matching, Bandit and Bearer adding heuristics, CodeQl and Pysa built on taint analysis, and Semgrep and Snyk combining techniques. Six vulnerability categories are drawn from the 2024 CWE Top 25, restricted to weaknesses with clearly identifiable trigger locations and excluding both memory-safety classes that do not arise in Python and logic-dependent classes such as missing authorization that lack a precise triggering statement: Command Injection (CWE-77), Deserialization of Untrusted Data (CWE-502), Code Injection (CWE-94), Cross-site Scripting (CWE-79), Path Traversal (CWE-22), and SQL Injection (CWE-89). Two benchmarks are constructed. The synthetic benchmark comprises 240 programs generated by DeepSeek-V3 consisting of 120 vulnerable programs, roughly twenty per category with each at least eighty lines long and containing a single vulnerability in one function, together with 12 patched counterparts produced by a follow-up repair prompt with vulnerability locations annotated at the method level. The real world benchmark was built by scanning 4,000 open-source Python projects yielding 151 candidate CVEs across 88 repositories, with manual verification against CVE and NCD records, proof of concept exploits and patch commits reduced to 108 confirmed vulnerabilities across 62 projects and 206 programs. Detection was assessed at method level, counting a report of the correct vulnerability type at a labeled location in the vulnerable version as a true position and any report on a patched version as a false positive; the authors this measure is a lower bound on real-world false positive rates, since alerts outside the known ground truth cannot be validated. The study had four significant findings. First, performance degrades sharply from synthetic to real-world code. Snyk leads the synthetic benchmark with an F1 of 79.5% but falls to 20.8% on real-world code, detecting 11.9% of vulnerabilities, Semgrep, the best real-world performer, still misses almost 60% and Pysa fails entirely, scoring zero on every metric. The authors conclude that results obtained from synthetic datasets may not generalize and can lead to misleading or contradictory conclusions. Secondly, coverage varies sharply by weakness class and tracks detection technique: pattern-matching and heuristic perform better on categories

involving misuse of recognizable APIs, while taint-based tools do better on cross-site scripting and path traversal, where distinguishing exploitable from safe cases require data flow reasoning. Command injection and code injection are the best covered categories at roughly 95% when all tools are combined while roughly 70% of cross-site scripting and SQL injection vulnerabilities go undetected by every tool Third, the tools and complementarity and combining them yields substantial gains: The Bandit-Bearer-CodeQL-Semgrep combination detects 38.9% more vulnerabilities than Bandit alone and 26% more than Semgrep, at the cost of a 10.8% precision reduction relative to CodeQL, with Dline contributing no uniquely detected vulnerabilities and CVE it flagged also reported by Bandit or Semgrp. However, even aggregating all either tools identifies only 66.7% of real-world vulnerabilities. Finally, a root cause analysis of every missed detection attributes failures to three categories: Incomplete or incorrect rule definitions, unresolved taint paths, and intrinsic limitation of the detection approach. This shows that rule deficiencies dominate for pattern-based tools, accounting for 93.5% of Bandit's misses and 87.2% of DevSkim's while taint-path resolution failures dominate for CodeQl at 70.3% Efficiency measurements show taint based tools to be the slowest, with Pysa's analysis time rising sharply beyond 690,000 lines of code.

b. **Datasets:**
    i. **Synthetic:** 240 Programs: 120 vulnerable (20 per CWE, > 80 lines, single vulnerability, DeepSeek-V3 571B) 120 patched. Exact generation prompt printed in Section 4.1
    ii. **Real-World:** 206 programs: 108 vulnerable CVEs from 62 projects, 98 patched
    iii. **CWE mapping:** 391 tool report messages mapped to CWE categories, released for reuse
c. **Github: https://github.com/Victor725/PySASTBench**

19. Tony, C., Díaz Ferreyra, N. E., Mutas, M., Dhif, S., & Scandariato, R. (2025). Prompting techniques for secure code generation: A systematic investigation. *ACM Transactions on Software Engineering and Methodology*, *34*(8), 1-53.
    a. Tony, Diaz Ferreyra, Mutas, Dhif, and Scandariato investigate whether prompting techniques affect the security of code that large language models generate from natural-language instructions. The study proceeds in two parts. A systematic literature review first establishes an inventory of prompting techniques applicable to code generation: from an initial pool of 358 papers exclusion criteria and two

round of backward snowballing yield 13 papers introducing distinct techniques, which the authors present as the first systematic inventory of prompting techniques suitable for code generation, observing that papers in the field typically make arbitrary selection of a few techniques based on convenience or on what other reference papers happen to use. A subset of the identified techniques is then adapted into seven reusable prompt templates and evaluated empirically. Four are zero shot variants that differ only in how security is signalled in the task instruction, a baseline carrying no security information, a naive-secure variant adding only the word “secure, a CWE-specific variant embedding mitigating guidance summarized from MITRE for the weakness the coding task targets, and a comprehensive variant asking the model to prevent the top weaknesses listed in CWE generally. The remaining three are zero shot chain of thought, Recursive Criticism and Improvement (RCI), in which the model is asked to review its own output for security problems and then revise it, and a personal template instructing the model to act as a software security expert. Each template is issued to GPT-3, GPT-3.5, and GPT-4 across 150 security-relevant coding tasks drawn from the LLMSecReval dataset, with generated code first checked for validity and task alignment before security assessment. Security is measured with Vandit as the primary analyzer and CodeQL as a supplementary check, and the authors are explicit about why both are used: absolute weakness counts differ between the tools because their detection approaches differ, so the analysis rests on relative difference between prompting techniques rather than absolute numbers. The two tools produced only minor differences in the rankings of techniques and agreed on the ebay and worst performers. Tool reliability is established independently rather than assumed: a sample of 315 Bandit results was manually inspected, and a reliability agreement test between the manual inspect and the tool out using a weight Cohen’s Kappa returned a value of 0.87 averaged across the seven techniques and three models, where values above 0.79 indicate strong agreement. Results are reported as weakness count, weakness rate per code snippet, and weakness density per line of code, allowing comparison across templates that produce programs of differing lengths. The findings confirm that LLM generated code contains a substantial number of security weaknesses, concentrated in CWE-78, CWE-259, CWE-94, and CWE-330, and show that prompting technique measurably reduces them, with the effect more pronounced the more

capable GPT-4. RCI emerged as the most effective technique in both languages tested, and the authors note that it had not previously been used in related work on secure code generation. Two elements of the study design are methodologically notable. Generalizability beyond Python was tested by repeating the evaluation on C code generated by GPT-4, with a separate 49-case manual sample returning a Kappa of 0.82. The authors address data contamination directly by observing that the LLMSecEval repository predates GPT-4's April 2023 training cutoff, then ran the Dolos source-code plagiarism detector over the RCI-generarated Python and obtained an average similarity score of 0.075 against LLMSecEval's reference implementations, well below the 0.5 threshold prior work associates with plagiarism, while noting that API-only interaction eliminates the risk of indirect leakage through reinforcement learning from human feedback.

b. **Datasets**
    i. **LLMSecEval:** 150 Natural language prompts specifying coding tasks that could lead to insecure implementations, each paired with a secure reference implementation
c. **Repository**
    i. **LLMSecEval:** https://github.com/tuhh-softsec/LLMSecEval
d. **Replication package:** https://figshare.com/s/195a75d8c4dd86816223
    i. Contains lit review results, techniques excluded from final selection with rational for each exclusion, and the code generated by all three models across all seven templates with validity and security analysis results

20. Bennett, G., Hall, T., Winter, E., & Counsell, S. (2024, June). Semgrep*: Improving the limited performance of static application security testing (sast) tools. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering* (pp. 614-623).
    a. Bennet, Hall, Winter and Counsell evaluate four SAST tools on production Java code with known vulnerabilities and then demonstrate that configuring a single tool can outperform a combination of tools running with their standard rule sets. Tool selection begins with OWASP's list of 114 source code analysis tools and applies successive filters, free or open source, Java-capable, deduplicated, actively developed within the past two years, security focused with CWE classification support, and over 1,000 Github stars as a proxy for developer adoption. This left CodeQl, Snyk Code, Semgrep, and FindSecBugs. The dataset

derived from the manually curated SAP dataset of Ponta et al., which had mined 205 open-source Java projects; the authors added the CWE identifiers by scraping the NVD and excluded vulnerabilities lacking CVE submission, unavailable repositories, uncheckable commits, or that no long build. This yielded 170 commits across 46 projects, 87 repositories, and 36 weakness types. Detection is verified manually instead of automatically, and the authors are open about why: the SAP dataset lacks vulnerable line locations, fix locations cannot be matched reliably to tool warnings because a fix may be applied in a different line, method, or file, and weakness types cannot be matched automatically because tools classify vulnerabilities inconsistently. A tool is credited with detection when its warning descriptions matches the NVD description and inspection of the source confirms it refers to the same instance; CWE labels are used for assistance but are not relied upon, as roughly half the vulnerabilities in the test data were labelled differently by a SAST tool than by the NVD. Because the dataset supports only recall, precision, and false positive rate are placed out of scope, with the authors focusing instead on false negatives on the grounds that missed vulnerabilities may matter more to developers. Four findings were discovered. First, individual detection rates range from 11.2% for Snyk Code to 26.5% for FindSecBugs, and combining all four reaches 38.8%. Detection varies sharply by weakness class: XXE (CWE-611) reaches 88.9% combined, path traversal 73.3%, and code injection 70%, while cross-site scripting reaches only 14.3%, improper input validation 0%, and neither infinite loops (CWE-835) nor race conditions (CWE-362) are detected by any tool at all. Secondly, the tools prove strong complementary, 53% of all detected vulnerabilities were found by exactly one tool, no tool includes another, and only five vulnerabilities were detected by all four. This led the authors to conclude that a suite of tools is required to achieve the best performance under a standard configuration. Finally, manual inspection of every false negative identifies whether each miss stemmed from a missing rule or an insufficient one, and the resulting patterns are encoded as 18 new and 11 edited Semgrep rules. The reconfigured tool, Semgrep* detects 44.7% of vulnerabilities, a 181% improvement over stock Semgrep, higher than the four tool combination and including 10 vulnerabilities no other tool found. The authors also observed a direct synthetic-versus-production discrepancy within a single weakness class, their tools detected 14.3% of cross-site scripting

vulnerabilities in production code, were SATE VI reported detection of up to 100% on 30 manually injected XSS vulnerabilities which they interpret as evidence that synthetic datasets do not represent variety of vulnerable code patterns found in production.

b. **Datasets**
    i. **Source:** The SAP dataset (Ponta et al. MSR 2019), manually curated from 205 open-source Java projects, 624 vulnerabilities with vulnerability identifier, repository URL and commit identifier.
    ii. **Ground Truth:** NVD-derived and lacks line-level locations, verification was manually performed by a single author, which the authors list as a construct validity threat
c. **Repository**
    i. **Replication package:** https://zenodo.org/records/10803730 per-tool detection results, combined results, suggested improvements, build commands for each repository version, the full CWE breakdown, which rules detect which vulnerabilities and the TAML files for the Semgrep* configuration

21. Mateo Tudela, F., Bermejo Higuera, J. R., Bermejo Higuera, J., Sicilia Montalvo, J. A., & Argyros, M. I. (2020). On combining static, dynamic and interactive analysis security testing tools to improve owasp top ten security vulnerability detection in web applications. *Applied Sciences*, *10*(24), 9119.
    a. Tudela, Bermejo Higuera, Bermejo Higuera, Sicili Montalvo, and Argyros investigate whether combining security testing tools of fundamentally different types improve vulnerability detection while controlling false positives. The study is distinguished from combination work by spanning three tool categories rather than one: static white-box analysis (SAST), Dynamic black-box analysis (DAST) and interactive white-box analysis (IAST), with two tools drawn from each: Fortify SCA and FindSecurityBugs for SATS, OWASP ZAP, Arachni for DAST, and Contract Community Edition and CxIAST for IAST. The authors motivate this by observing that the techniques and algorithms underlying each tool type differ, so each detects a different subsect of vulnerability classes and exhibits a different false positive profile, and that these differences may produce exploitable synergies when tools are combined. Benchmark selection follows explicit criteria: A test bench must be credible, portable, representative, require minimal

modification, and allow all tools to execute under identical conditions. The authors survey several candidates including Wavsep, Securebench, Micro, the NIST SAMATE suites, Delta-bench, and the OWASP Top Ten Benchmark before settling on the OWASP Benchmark project, as it is the only candidate analyzable by all three tool types. From it, they randomly select 320 Java test cases distributed across nine weakness classes and balanced so that exactly half are true positives and half are false positives, a design that makes precision and false positive rates directly measurable rather than approximable. All tools are run with default configuration. The evaluation applies a wider metric set than most tool-comparision studies: precision, false positive rate, F-measure, and F which varied to weigh precision or recall differently, alongside Markedness and other classification measures. The Parameterization serves a specific purpose, the authors assess combinations against three levels of web application criticality, on the reasoning that an organization running a high-criticality application should weigh recall more heavily than one running a low-criticality application so that no single ranking is appropriate for every deployment context. Combinations of two and three tools are then ranked under each weighting.